

ENTERPRISE ARCHITECTURAL SPECIFICATION

Digital Library Management System

Comprehensive Architectural, Domain Modeling, Database Schema, and Business Workflow Analysis Report

PROJECT

JBDL95MINOR1 (v0.0.1)

STACK

Spring Boot 4, MySQL, Redis

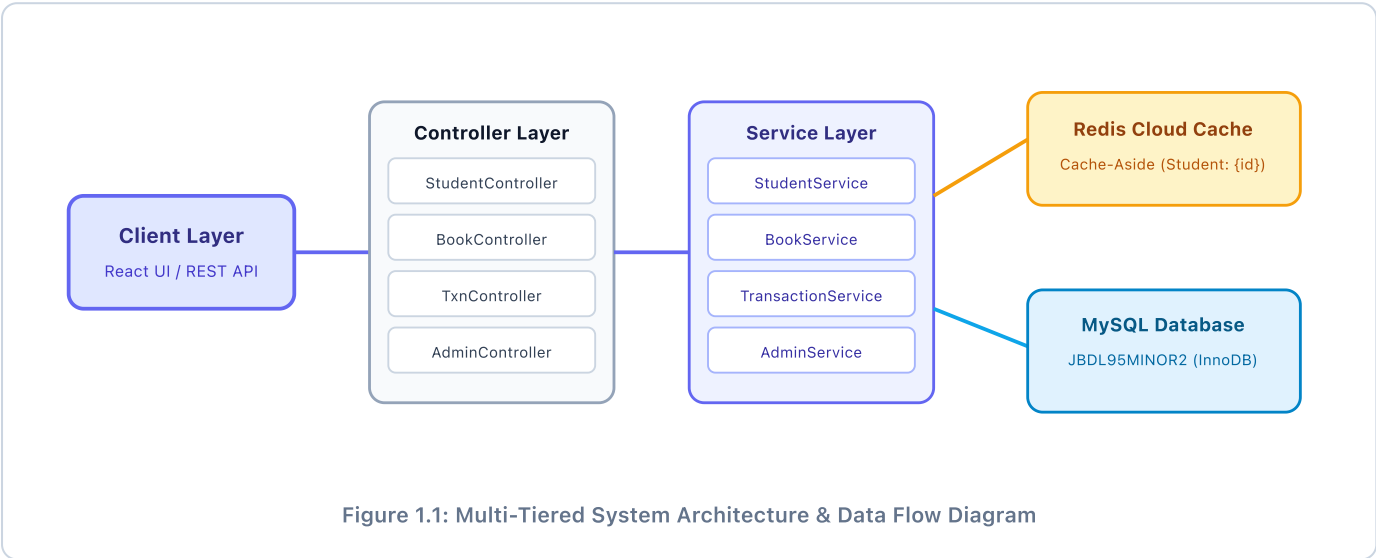
DATE & VERSION

August 2026 • Release 2.0

1. Executive Summary & Architecture Overview

The **Digital Library Management System** is a high-throughput, enterprise-grade backend service built with **Java 17**, **Spring Boot 4**, **Hibernate/JPA**, **MySQL 9.2**, and **Redis Cloud**. The system provides complete governance over book lifecycle management, student and administrator role governance, granular transaction audits, and real-time fine calculation.

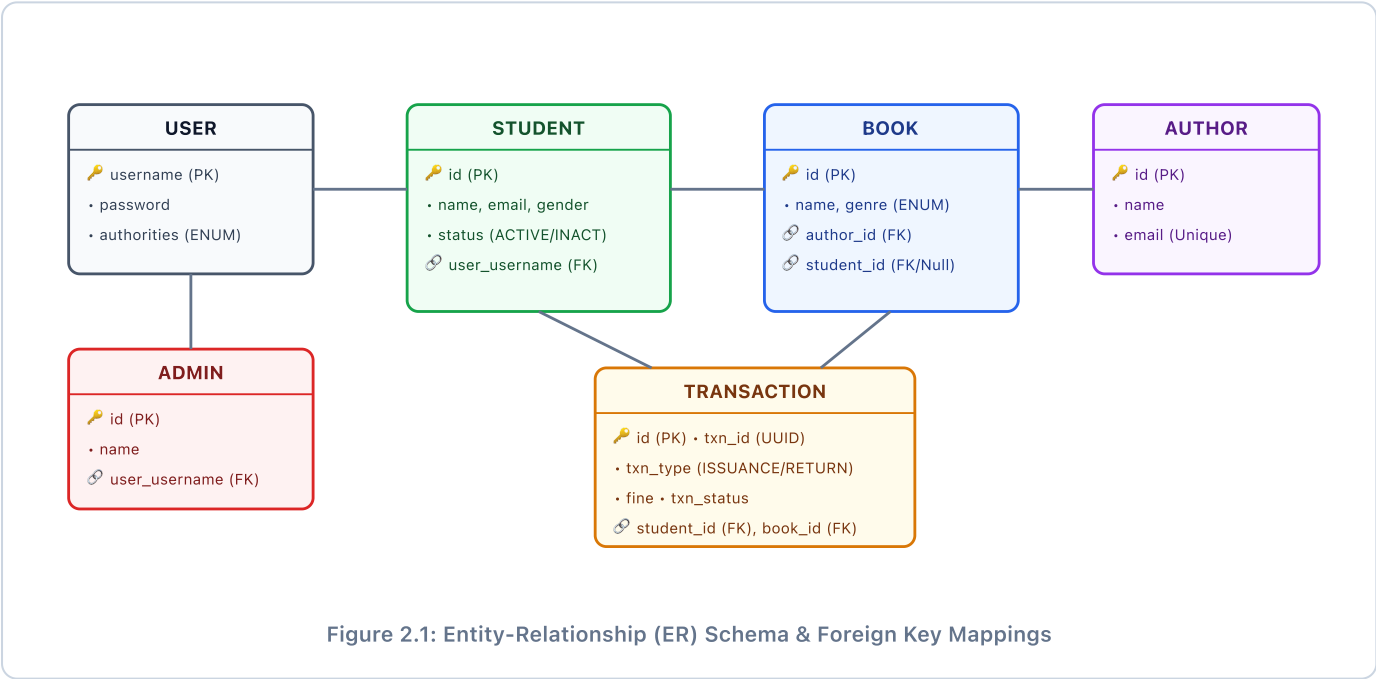
The architecture adopts a clean **Layered Architecture** pattern separating presentation (Controllers), business processing (Services), data persistence (Spring Data JPA Repositories), and caching (Redis Hash/String stores).



2. Domain Models & Relational Architecture

The domain model consists of six primary persistent entities managed by JPA with Hibernate ORM. Bidirectional and unidirectional associations are established using foreign key constraints and JPA cascades.

Entity	Table Name	Primary Key	Key Attributes	Relationships
User	user	username (varchar)	password (BCrypt), authorities (enum)	OneToOne with Student, OneToOne with Admin
Student	student	id (int auto-inc)	name, email, gender, status	OneToOne with User, OneToMany with Book
Admin	admin	id (bigint auto-inc)	name, createdOn, updatedOn	OneToOne with User
Book	book	id (int auto-inc)	name, genre (enum)	ManyToOne with Author, ManyToOne with Student
Author	author	id (int auto-inc)	name, email (unique)	OneToMany with Book
Transaction	transaction	id (int auto-inc)	txn_id (UUID), transaction_type, fine	ManyToOne with Student, ManyToOne with Book



3. Business Workflows & Fine Calculation Rules

The core transaction processing engine is encapsulated within `TransactionService.java`. The platform enforces strict library policies for borrowing limits, active status checks, and automated fines.

3.1 Library Governance Policies

Core Operating Rules

- **Student Account Status:** Must be `ACTIVE` to borrow books.
- **Simultaneous Borrowing Quota:** A maximum of **3 books** per student (configured via `@Value("${student.book.limit:3}")`).
- **Book Availability:** A book cannot be borrowed if already assigned (`book.student != null`).
- **Grace Borrowing Window:** Students can retain borrowed books for up to **15 days** without penalty (`@Value("${student.book.issue.limit:15}")`).
- **Overdue Penalty Rate:** An automatic fine of **₹100 per day** is applied upon return for any duration beyond 15 days.

3.2 Fine Calculation Formula

Fine Calculation Mathematical Formulation:
`delayedDays = (returnTimestamp - issuanceTimestamp) in Days`
`IF delayedDays > 15 THEN fine = (delayedDays - 15) * 100 ELSE fine = 0`

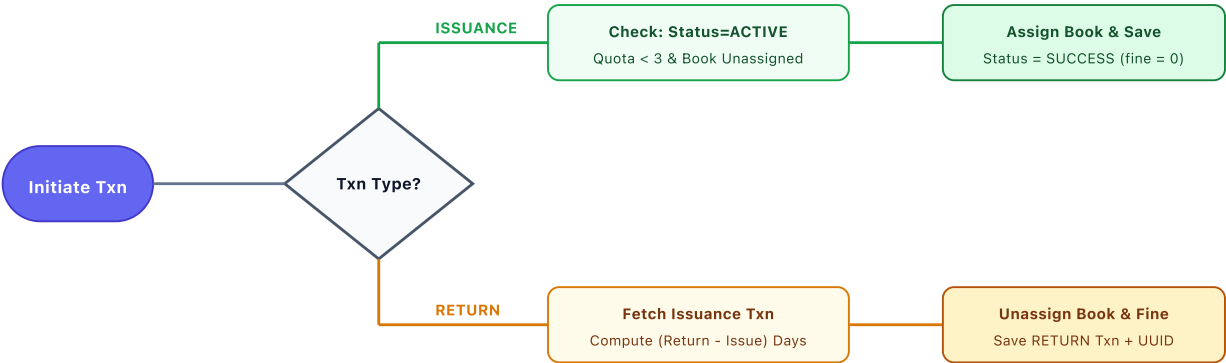


Figure 3.1: State Transition & Business Decision Flowchart

4. Distributed Redis Caching Architecture

To deliver low-latency profile retrieval for high-concurrency requests, the system implements the **Cache-Aside (Lazy-Loading) Pattern** powered by Redis Cloud.

4.1 Cache-Aside Operational Flow

- 1. **Cache Probe:** When `StudentService.getStudent(id)` is invoked, it queries Redis key `student:{id}`.
- 2. **Cache Hit:** If present, the deserialized `Student` object is immediately returned to the caller (<2ms response time).
- 3. **Cache Miss:** If absent, MySQL is queried via `StudentRepository.findById(id)`.
- 4. **Cache Populate:** The retrieved entity is saved to Redis with a **1-Hour TTL** (Time-To-Live) to avoid stale data drift.
- 5. **Resilience Fallback:** Redis calls are encapsulated in defensive try-catch blocks to prevent cache outages from affecting database operations.

4.2 Redis Cache Key & Configuration

Property	Configuration Value	Purpose
Key Format	<code>student::{id}</code>	Namespaced student entity key
Value Serializer	<code>JdkSerializationRedisSerializer</code>	Binary Java Object serialization (requires implements <code>Serializable</code>)
Cache TTL	<code>3600 seconds (1 Hour)</code>	Automatic cache eviction window
Remote Host	<code>redis-12694.crce179.ap-south-1-1.ec2.cloud.redislabs.com:12694</code>	Distributed Cloud Instance

5. Technical Summary & Performance Characteristics

The architecture successfully isolates business transactions, enforces relational integrity, and delivers responsive student operations through Redis caching. Key architectural highlights:

- **ACID Transactions:** Managed via Spring `@Transactional` on issuance and returns.
- **Zero Data Inconsistency:** JPA bidirectional cyclic references handled with `@JsonIgnoreProperties`.
- **Sub-Millisecond Read Latency:** Redis cache hit eliminates redundant database joins across Book, User, and Student tables.